МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ «ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №1
по дисциплине: «Объектно-ориентированное программирование»
Тема: «Реализация игры с пошаговым перемещением игрока и сражением с врагами»

Студент гр. 5382 Рязанов А.Д.

Преподаватель Шестопалов Р.П.

Санкт-Петербург
2026

СОДЕРЖАНИЕ

ЦЕЛЬ РАБОТЫ 3

ЗАДАНИЕ 4

1 ОПИСАНИЕ АРХИТЕКТУРЫ 5

1.1 UML-диаграмма классов 5

1.2 Текстовое описание архитектуры и обоснование архитектурных решений 6

1.3 Используемые паттерны проектирования 7

2 ОПИСАНИЕ КЛАССОВ 9

2.1 Класс Robot 9

2.2 Класс Cell 10

2.3 Класс Field 11

2.4 Класс GameEngine 11

2.5 Класс Visualizer 13

2.6 Перечисление Config 13

3 ТЕСТИРОВАНИЕ 15

3.1 Методика тестирования 15

3.2 Результаты тестирования 15

ЗАКЛЮЧЕНИЕ 18

ЦЕЛЬ РАБОТЫ

Целью лабораторной работы является применение принципов объектно-ориентированного программирования на языке C++ при разработке пошаговой игры с перемещением роботов по игровому полю и взаимодействием с игровыми сущностями. В рамках работы необходимо построить предметную модель, инкапсулировать состояние объектов, разделить игровую логику и визуализацию, а также обеспечить возможность дальнейшего расширения программы.

ЗАДАНИЕ

В лабораторной работе №1 необходимо реализовать следующие компоненты игры:

1. Класс робота (Robot), включающий характеристики: здоровье, максимальное здоровье, показатель атаки, энергия, максимальная энергия, опыт и порог опыта для повышения ранга. Реализовать методы изменения характеристик с контролем допустимых значений, метод взаимодействия с другими роботами (атака врага / лечение союзника), а также механизм повышения ранга с увеличением характеристик при накоплении опыта.

2. Класс игрового поля (Field), состоящего из клеток. Размер поля задаётся аргументами конструктора с контролем допустимых границ. Поле хранит информацию о проходимости клеток, непроходимые клетки блокируют перемещение роботов.

3. Систему перемещения роботов по полю с учётом проходимости клеток. При попытке переместиться на клетку, занятую другим роботом, инициируется взаимодействие. Реализовать учёт скорости робота и сложности проходимости клеток.

4. Игровой цикл с пошаговыми ходами: ход игрока, ход вражеских роботов под управлением компьютера, восстановление энергии в конце хода. Уровень считается пройденным при уничтожении всех вражеских роботов.

5. Систему консольной визуализации поля, отображающую проходимые и непроходимые клетки, расположение робота игрока, вражеских роботов и завода. Визуализация реализуется отдельным классом, не зависящим от игровых сущностей.

Дополнительно реализованы: характеристика скорости робота с зависимостью от сложности проходимости клеток и система видимости (робот видит ограниченную область вокруг себя).

1 ОПИСАНИЕ АРХИТЕКТУРЫ

1.1 UML-диаграмма классов

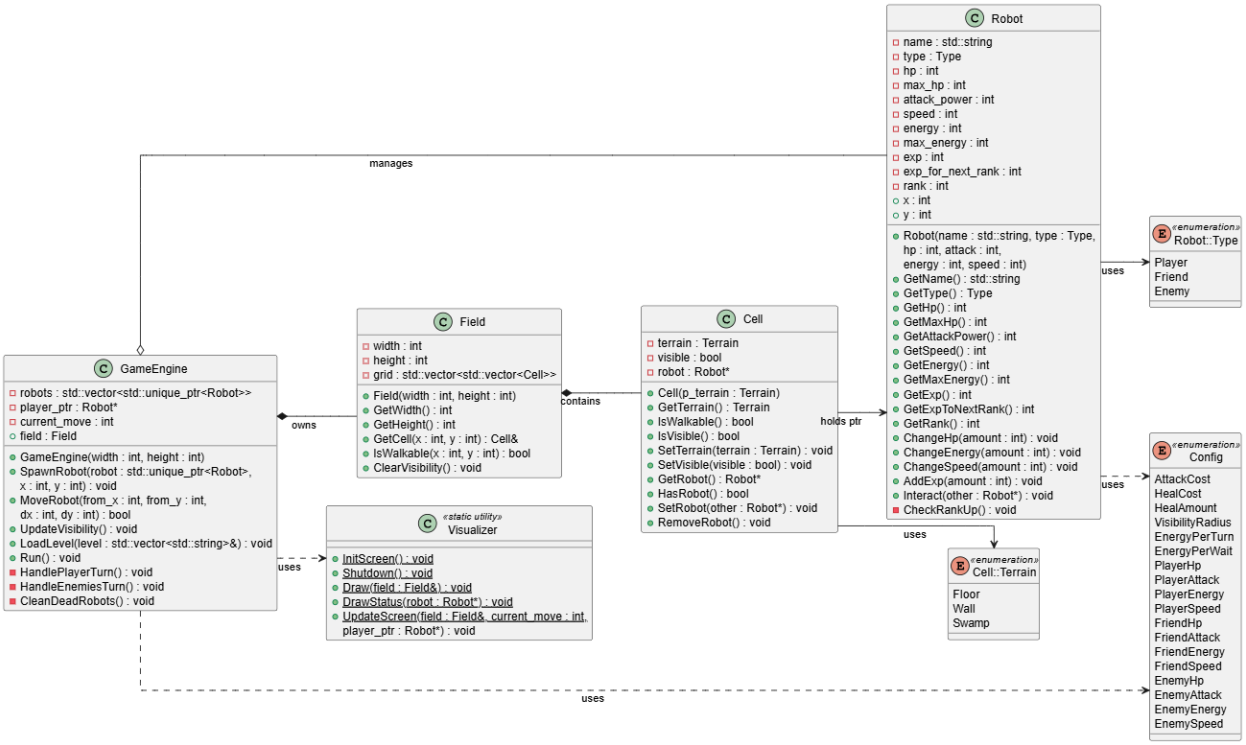


Рисунок 1 — UML-диаграмма классов

На диаграмме отражены следующие классы и их отношения:

1. **Robot** — класс, моделирующий робота. Содержит характеристики (hp, max_hp, attack_power, energy, max_energy, exp, exp_to_next_rank, rank, speed), публичные координаты x, y, методы доступа (get/set) и метод взаимодействия Interact().
2. **Cell** — класс, описывающий отдельную клетку поля. Хранит тип местности (Terrain), флаг видимости (visible) и указатель на робота (Robot*).
3. **Field** — класс игрового поля. Агрегирует двумерный вектор структур Cell. Предоставляет методы доступа к размеру и клеткам, проверки проходимости IsWalkable() и очистки видимости ClearVisibility().
4. **GameEngine** — класс, объединяющий игровое поле и роботов. Содержит объект Field, вектор std::unique_ptr<Robot>, указатель на

игрока player_ptr. Предоставляет методы спавна роботов, перемещения, обновления видимости, загрузки уровня и запуска игрового цикла.

5. **Visualizer** — класс визуализации. Реализует статические методы для инициализации ncurses, отрисовки поля и панели статуса робота.
6. **Config** — перечисление с константами баланса игры: цены действий, эффекты, параметры роботов, радиус видимости, восстановление энергии. Используется классами Robot и GameEngine.
7. **Robot::Type** — перечисление типов роботов: Player, Friend, Enemy.
8. **Cell::Terrain** — перечисление типов местности: Floor, Wall, Swamp.

Основные отношения на диаграмме:

- GameEngine ◆— Field — композиция: движок владеет полем по значению.
- GameEngine ◇— Robot — агрегация: движок управляет коллекцией роботов через unique_ptr.
- Field ◆— Cell — композиция: поле владеет клетками по значению.
- Cell —> Robot — ассоциация: клетка держит невладеющий указатель на робота.
- Robot —> Type — ассоциация с перечислением типа.
- Cell —> Terrain — ассоциация с перечислением местности.
- Robot ⇝ Config — зависимость.
- GameEngine ⇝ Visualizer — зависимость (использует статические методы отрисовки).

1.2 Текстовое описание архитектуры и обоснование архитектурных решений

Архитектура программы построена по модульному принципу с чётким разделением функционала игры между классами.

Игровые сущности — класс Robot и класс Cell. Данный слой инкапсулирует состояние отдельных игровых объектов. Robot хранит характеристики и предоставляет контролируемый доступ к ним через get-

методы. Класс Cell описывает свойства одной клетки поля (тип местности, видимость, наличие робота).

Игровое поле — класс Field. Field реализует двумерную сетку клеток с контролем допустимых размеров (от 10×5 до 60×20). Поле хранит клетки по значению в std::vector<std::vector<Cell>>.

Игровая логика — класс GameEngine. GameEngine реализует игровой цикл (ход игрока, ход врагов, обновление видимости, восстановление энергии) и условия победы/поражения. Логика перемещения реализована как приватный метод MoveRobot, поскольку оперирует состоянием движка. Спавн роботов и загрузка уровня также вынесены в движок.

Визуализация — класс Visualizer. Реализован отдельно от игровых сущностей, что соответствует требованию: «классы робота и поля не должны выводить ничего и знать какую-либо информацию о том, как должны отрисовываться». Visualizer собирает информацию из Field и Robot и отображает её в консоли с учётом системы видимости.

Константы баланса — перечисление Config. Вынесено в отдельный заголовочный файл, чтобы убрать магические числа из логики Robot и GameEngine и обеспечить единое место для настройки баланса игры.

Инварианты основных классов поддерживаются через инкапсуляцию. Здоровье Robot всегда ограничивается диапазоном [0, max_hp] методом ChangeHp, энергия — [0, max_energy] методом ChangeEnergy. Скорость не может быть меньше 1 (метод ChangeSpeed). Позиция робота (x, y) синхронизируется с указателем в Cell при каждом перемещении. CleanDeadRobots удаляет мёртвых роботов и обнуляет соответствующие указатели в Cell.

1.3 Используемые паттерны проектирования

В архитектуре разработанной программы прослеживаются следующие паттерны проектирования:

Композиция. Field хранит std::vector<std::vector<Cell>>, а GameEngine владеет объектом Field по значению. Время жизни клеток определяется временем жизни поля, время жизни поля — временем жизни движка.

Агрегация. GameEngine управляет коллекцией роботов через std::vector<std::unique_ptr<Robot>>. Роботы создаются отдельно (через make_unique) и передаются движку.

Разделение ответственности. Cell и Field отвечают за данные игрового поля, Robot — за состояние и поведение игровой единицы, GameEngine — за игровой цикл и правила, Visualizer — за представление, Config — за константы баланса.

Visualizer полностью отделён от Robot и Field — ни один из них не содержит методов вывода. Это позволяет менять способ отображения без изменения правил игры.

Инкапсуляция и сохранение инвариантов. Приватные поля изменяются через методы, а операции изменения здоровья, энергии и скорости контролируют допустимые границы (через std::clamp).

RAII и уникальное владение. Роботы хранятся через std::unique_ptr, что даёт автоматическое освобождение памяти и чёткое определение владельца (GameEngine).

2 ОПИСАНИЕ КЛАССОВ

2.1 Класс Robot

Класс Robot моделирует робота с набором характеристик и поведенческих методов. Определён в файлах Robot.h и Robot.cpp. Тип робота задаётся вложенным перечислением Robot::Type со значениями Player, Friend, Enemy.

Приватные поля класса:

- name (std::string) — имя робота.
- type (Type) — тип робота: Player, Friend или Enemy.
- hp (int) — текущее здоровье. Не может быть отрицательным.
- max_hp (int) — максимальное здоровье. Увеличивается при повышении ранга.
- attack_power (int) — показатель атаки. Определяет величину наносимого урона.
- speed (int) — скорость перемещения. Определяет количество шагов за ход с учётом сложности проходимости клеток.
- energy (int) — текущая энергия. Восстанавливается в конце каждого хода.
- max_energy (int) — максимальная энергия.
- exp (int) — текущий опыт. Увеличивается на 5 при каждой успешной атаке.
- exp_to_next_rank (int) — порог опыта для повышения ранга (начальное значение 50).
- rank (int) — текущий ранг (начальное значение 1).

Публичные поля:

- x, y (int) — координаты робота на игровом поле.

Методы взаимодействия:

Метод Interact(Robot* other) реализует взаимодействие с другим роботом. Если типы различаются и один из них Player, а другой Enemy — выполняется атака: цель теряет attack_power здоровья, атакующий теряет

Config::AttackCost энергии и получает 5 опыта. Если типы Player и Friend — выполняется лечение: цель получает Config::HealAmount здоровья, друг теряет Config::HealCost энергии.

Метод ChangeHp(int amount) изменяет здоровье с контролем границ [0, max_hp] через std::clamp. Аналогично работает ChangeEnergy с границами [0, max_energy].

Метод AddExp(int amount) накапливает опыт и вызывает CheckRankUp() при достижении порога. При повышении ранга: max_hp += 25, max_energy += 25, attack_power += 5, здоровье и энергия восстанавливаются до максимума, exp_to_next_rank *= 2.

Контроль инвариантов: класс гарантирует, что здоровье всегда в диапазоне [0, max_hp], энергия — в [0, max_energy], скорость не меньше 1, а тип робота однозначно определяет его поведение при взаимодействии.

2.2 Класс Cell

Класс **Cell** представляет одну клетку игрового поля. Определён в файлах Cell.h и Cell.cpp. Тип местности задаётся вложенным перечислением Cell::Terrain со значениями Wall, Floor, Swamp.

Приватные поля:

- **terrain** (Terrain) — тип местности: Floor (пол), Wall (стена) или Swamp (болото). По умолчанию Floor.
- **visible** (bool) — флаг видимости клетки. По умолчанию false.
- **robot** (Robot*) — указатель на робота, стоящего на клетке. По умолчанию nullptr.

Методы:

- GetTerrain() — возвращает тип местности.
- IsWalkable() — возвращает true, если клетка проходима (не стена) и на ней нет робота.
- IsVisible() — возвращает true, если клетка в зоне видимости.
- SetTerrain(Terrain), SetVisible(bool) — изменяют соответствующие поля.

- GetRobot(), HasRobot() — доступ к указателю на робота.
- SetRobot(Robot*), RemoveRobot() — устанавливают и сбрасывают указатель.

2.3 Класс Field

Класс Field (файлы Field.h, Field.cpp) реализует двумерное игровое поле как матрицу объектов Cell.

Приватные поля:

- width (int), height (int) — размеры поля.
- grid (std::vector<std::vector<Cell>>) — двумерный вектор клеток.

Ключевые проектные решения:

1. **Контроль размеров.** Конструктор Field(int width, int height) проверяет допустимость аргументов: ширина от 10 до 60, высота от 5 до 20. При выходе за границы выбрасывается std::invalid_argument.
2. **Двумерный вектор.** Поле хранится как std::vector<std::vector<Cell>>. Клетки создаются в конструкторе через resize.
3. **Доступ к данным.** Метод GetCell(x, y) возвращает неконстантную ссылку на клетку — это сделано намеренно, чтобы GameEngine мог модифицировать проходимость и видимость клеток. Для безопасного чтения предусмотрен метод IsWalkable(x, y).
4. **Управление видимостью.** Метод ClearVisibility() сбрасывает флаг visible у всех клеток, кроме стен — вызывается перед каждым обновлением зоны видимости.

2.4 Класс GameEngine

Класс GameEngine (файлы GameEngine.h, GameEngine.cpp) объединяет игровое поле, коллекцию роботов и реализует игровой цикл.

Приватные поля:

- robots (std::vector<std::unique_ptr<Robot>>) — владеет всеми роботами.
- player_ptr (Robot*) — наблюдение за игроком для быстрого доступа.

Публичные поля:

- field (Field) — игровое поле.

Основные методы:

- GameEngine(int width, int height) — конструктор, инициализирует поле через список инициализации.
- SpawnRobot(std::unique_ptr<Robot>, int x, int y) — добавляет робота на поле, если клетка проходима. Если робот имеет тип Player, запоминает player_ptr.
- MoveRobot(int from_x, int from_y, int dx, int dy) — реализует перемещение робота с учётом его скорости. При столкновении с другим роботом инициирует Interact. При выходе из болота снижает скорость на 1.
- UpdateVisibility() — раскрывает клетки в радиусе Config:: VisibilityRadius от игрока.
- LoadLevel(std::vector<std::string>&) — разбирает строковое представление уровня: # → стена, ~ → болото, R → игрок, F → друг, E → враг. Создаёт роботов через std::make_unique.
- Run() — главный игровой цикл. Выполняет: очистку видимости, обновление видимости, отрисовку, ход игрока, удаление мёртвых, ход врагов, повторное удаление мёртвых, обновление видимости, восстановление энергии.

Приватные методы:

- HandlePlayerTurn() — читает клавишу (getch), преобразует в смещение и вызывает MoveRobot. Enter восстанавливает 10 энергии, q — выход.
- HandleEnemiesTurn() — для каждого врага вычисляет направление к игроку и вызывает MoveRobot.
- CleanDeadRobots() — удаляет роботов с hp <= 0, сбрасывает указатели в Cell и обнуляет player_ptr при смерти игрока.

2.5 Класс Visualizer

Класс Visualizer (файлы Visualizer.h, Visualizer.cpp) — класс визуализации. Все методы статические.

Методы:

- InitScreen() — инициализирует ncurses: initscr, cbreak, noecho, keypad, curs_set(0), clear.
- Shutdown() — вызывает endwin() для восстановления терминала.
- Draw(Field&) — обходит все клетки поля и выводит символы: . для невидимых, # для стены, R/F/E для роботов, ~ для болота, пробел для пустой клетки.
- DrawStatus(Robot*) — выводит панель статуса: имя, HP, EN, ATK, SPD, RANK, EXP, POS.

Класс полностью отделён от модели: ни Field, ни Cell, ни Robot не знают о способе отображения.

2.6 Перечисление Config

Перечисление Config (файл Config.h) задаёт все числовые константы баланса игры:

Цены действий:

- AttackCost = 20 — энергия на атаку.
- HealCost = 20 — энергия на лечение.

Эффекты действий:

- HealAmount = 15 — здоровье, восстанавливаемое при лечении.

Игровой цикл:

- VisibilityRadius = 3 — радиус обзора.
- EnergyPerTurn = 10 — восстановление энергии за ход.
- EnergyPerWait = 10 — энергия за пропуск хода (Enter).

Параметры игрока:

- PlayerHp = 100, PlayerAttack = 20, PlayerEnergy = 60, PlayerSpeed = 2.

Параметры друга:

- FriendHp = 30, FriendAttack = 5, FriendEnergy = 20, FriendSpeed = 0.

Параметры врага:

- EnemyHp = 50, EnemyAttack = 10, EnemyEnergy = 30, EnemySpeed = 1.

Используется классами Robot и GameEngine вместо магических чисел. Вынос в отдельный файл позволяет изменять баланс игры в одном месте.

3 ТЕСТИРОВАНИЕ

3.1 Методика тестирования

Для автоматического тестирования использован фреймворк GoogleTest. Тесты проверяют публичное поведение основных классов и типичные взаимодействия между ними. Каждый тест создаёт минимально необходимое состояние, выполняет одну операцию и сравнивает фактический результат с ожидаемым при помощи макросов EXPECT_EQ, EXPECT_TRUE, EXPECT_FALSE, EXPECT_THROW, EXPECT_NO_THROW.

Тестовый набор разделён на группы: RobotConstructor, RobotChangeHp, RobotChangeEnergy, RobotChangeSpeed, RobotExp, RobotInteract, CellConstructor, CellTerrain, CellRobot, CellVisibility, FieldConstructor, FieldDimensions, FieldCells, FieldVisibility, GameEngineSpawn, GameEngineLoadLevel, GameEngineMove, GameEngineVisibility.

Отдельно проверяются инварианты характеристик робота, корректность создания и загрузки уровня, блокировка непроходимых клеток, взаимодействие с вражескими и дружественными роботами, эффект болота, система видимости.

3.2 Результаты тестирования

```
[==========] Running 21 tests from 4 test suites.
[----------] Global test environment set-up.
[----------] 5 tests from Robot
[ RUN      ] Robot.ConstructorSetsValues
[       OK ] Robot.ConstructorSetsValues (0 ms)
[ RUN      ] Robot.ChangeHpClampsToRange
[       OK ] Robot.ChangeHpClampsToRange (0 ms)
[ RUN      ] Robot.RankUpIncreasesStats
[       OK ] Robot.RankUpIncreasesStats (0 ms)
[ RUN      ] Robot.PlayerAttacksEnemy
[       OK ] Robot.PlayerAttacksEnemy (0 ms)
[ RUN      ] Robot.PlayerNotAttacksWithoutEnergy
[       OK ] Robot.PlayerNotAttacksWithoutEnergy (0 ms)
```

```
[----------] 5 tests from Robot (0 ms total)

[----------] 4 tests from Cell
[ RUN      ] Cell.DefaultIsFloorAndWalkable
[       OK ] Cell.DefaultIsFloorAndWalkable (0 ms)
[ RUN      ] Cell.WallIsNotWalkable
[       OK ] Cell.WallIsNotWalkable (0 ms)
[ RUN      ] Cell.RobotBlocksWalkable
[       OK ] Cell.RobotBlocksWalkable (0 ms)
[ RUN      ] Cell.VisibilityCanBeChanged
[       OK ] Cell.VisibilityCanBeChanged (0 ms)
[----------] 4 tests from Cell (0 ms total)

[----------] 5 tests from Field
[ RUN      ] Field.ValidSize
[       OK ] Field.ValidSize (0 ms)
[ RUN      ] Field.InvalidSizeThrows
[       OK ] Field.InvalidSizeThrows (0 ms)
[ RUN      ] Field.CellsStartAsFloor
[       OK ] Field.CellsStartAsFloor (0 ms)
[ RUN      ] Field.WallBlocksWalkable
[       OK ] Field.WallBlocksWalkable (0 ms)
[ RUN      ] Field.ClearVisibilityResets
[       OK ] Field.ClearVisibilityResets (0 ms)
[----------] 5 tests from Field (0 ms total)

[----------] 7 tests from GameEngine
[ RUN      ] GameEngine.SpawnRobotOnEmptyCell
[       OK ] GameEngine.SpawnRobotOnEmptyCell (0 ms)
[ RUN      ] GameEngine.SpawnRobotOnWallDoesNothing
[       OK ] GameEngine.SpawnRobotOnWallDoesNothing (0 ms)
[ RUN      ] GameEngine.LoadLevelParsesSymbols
[       OK ] GameEngine.LoadLevelParsesSymbols (0 ms)
[ RUN      ] GameEngine.MoveRobotMovesPlayer
[       OK ] GameEngine.MoveRobotMovesPlayer (0 ms)
```

```
[ RUN      ] GameEngine.MoveRobotStopsAtWall
[       OK ] GameEngine.MoveRobotStopsAtWall (0 ms)
[ RUN      ] GameEngine.MoveRobotAttacksEnemy
[       OK ] GameEngine.MoveRobotAttacksEnemy (0 ms)
[ RUN      ] GameEngine.UpdateVisibilityRevealsAroundPlayer
[       OK ] GameEngine.UpdateVisibilityRevealsAroundPlayer (0 ms)
[----------] 7 tests from GameEngine (0 ms total)

[----------] Global test environment tear-down
[==========] 21 tests from 4 test suites ran. (0 ms total)
[  PASSED  ] 21 tests.
```

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы была разработана объектно-ориентированная игра на языке C++. Реализованы классы Cell и Field для представления игрового поля, класс Robot для описания игровой единицы с системой рангов, класс GameEngine для управления состоянием игрового мира и игровым циклом, класс Visualizer для консольной визуализации через ncurses, перечисление Config для хранения констант баланса.

В архитектуре использованы композиция, агрегация, инкапсуляция и уникальное владение через std::unique_ptr. Функциональность разделена между классами по ответственности. Робот имеет здоровье, энергию, атаку, опыт, скорость и ранг; реализовано контролируемое изменение характеристик и повышение ранга при накоплении опыта. Перемещение учитывает границы поля, проходимость клеток и бюджет скорости; при столкновении используется взаимодействие между роботами. Болото снижает скорость робота. Система видимости ограничивает область обзора игрока радиусом 3.

Для автоматической проверки реализован набор тестов GoogleTest, охватывающий основные классы, создание уровня, перемещение, взаимодействия и систему видимости. Тесты собираются через CMake и запускаются как через прямой вызов исполняемого файла, так и через CTest.